

Formalized Cellular Automata Theory in Lean 4

This project formalizes key results about cellular automata that recognize languages, focusing on **real-time** recognition, **one-way (left-independent) CAs**, and **advice mechanisms**.

Setup and Non-Standard Definitions

Cellular Automaton

A CA is a one-dimensional cellular automaton with radius-1 neighborhood, given as a tuple $C = (Q, \Sigma, \Gamma, \delta, \text{embed}, \text{project})$ with state set Q , input alphabet Σ , output alphabet Γ , local transition $\delta : Q^3 \rightarrow Q$, and maps $\text{embed} : \Sigma \rightarrow Q$, $\text{project} : Q \rightarrow \Gamma$. The split into input/output types lets CAs act as transducers.

A **configuration** is a map $c : \mathbb{Z} \rightarrow Q$. One step: $\text{next}(c)_p = \delta(c_{p-1}, c_p, c_{p+1})$. We write $\Delta_C^t(c)$ for the t -fold iterate, and $\text{comp}_C(c, t, i) = \text{project}(\Delta_C^t(\text{embed} \circ c)_i)$.

Word Embedding (0-indexed)

Words are embedded into configurations with **0-based indexing**: a word w of length n occupies positions $0, 1, \dots, n-1$, with all other positions set to the border symbol $\#$. Formally:

$$\langle w \rangle(p) = \begin{cases} w_p & \text{if } 0 \leq p < |w| \\ \# & \text{otherwise} \end{cases}$$

For language-recognizing CAs the input alphabet is $\Sigma_\# = \Sigma \cup \{\#\}$, so $\text{embed}(\#)$ gives the border state. Note that in this formalization, the border state has **no a priori constraints** — it need not be quiescent or dead. This is more general than many textbook definitions, which assume $\delta(\#, \#, \#) = \#$. Results 4 and 5 below show that a passive or dead border can always be imposed without changing the recognized language, so this generalization is conservative and the language classes agree with the standard ones.

Trace

The **trace** of C on configuration c is the temporal output sequence at position 0:

$$\text{trace}_C(c) : \mathbb{N} \rightarrow \Gamma, \quad t \mapsto \text{comp}_C(c, t, 0)$$

Real-Time Trace

The **real-time trace** is the word-to-word transduction where position i reads out time i :

$$\text{trace_rt}_C(w) = (\text{trace}_C(\langle w \rangle)(0), \text{trace}_C(\langle w \rangle)(1), \dots, \text{trace}_C(\langle w \rangle)(n-1))$$

This is the central notion for composing CA transducers: $\text{trace_rt}_C : \Sigma^* \rightarrow \Gamma^*$ is a length-preserving map.

Left-Independent (One-Way) CA

A CA is **left-independent** if δ ignores its left argument:

$$\forall a, a', b, c : \delta(a, b, c) = \delta(a', b, c)$$

These correspond to **one-way CAs (OCA)**. The **left-independent light cone** at position p and time t for a word of length n is $\{p \text{ mid} - t \leq p < n\}$.

Real-Time Language Class $\mathcal{L}(\text{CA}_{\text{rt}})$

A CA **accepts** a word w of length n by reading a designated cell at a designated time. A **timed CA** specifies functions $t(n)$ (time) and $p(n)$ (position) and accepts w iff $\text{comp}_C(\langle w \rangle, t(|w|), p(|w|)) = \text{true}$.

For the standard classes:

- **CA**: read position 0, i.e. $p(n) = 0$.
- **CA_{rt}**: read position 0 at time $n - 1$ (real-time).
- **OCA**: left-independent CA reading at position 0.
- **OCA_{rt}**: left-independent, real-time.

The class $\mathcal{L}(\text{CA}_{\text{rt}})$ is the set of languages recognized by real-time CAs. Note the 0-indexed embedding: a word of length n occupies positions $0, \dots, n - 1$, and at time $n - 1$ the information from the rightmost cell has just reached position 0.

Advice Functions

An **advice** is a length-preserving map $f : \Sigma^* \rightarrow \Gamma^*$ with $|f(w)| = |w|$.

- **RT-closed**: f is RT-closed if $\mathcal{L}(\text{CA}_{\text{rt}}(\Sigma \times \Gamma)/f) = \mathcal{L}(\text{CA}_{\text{rt}}(\Sigma))$, i.e. the advice does not increase the power of real-time CAs.
- **Prefix-stable**: $f(w_{[0..i]}) = f(w)_{[0..i]}$ for all w, i .
- **Two-stage**: f factors as $f = M \circ \text{trace_rt}_C$, where C is a CA real-time transducer and M is a finite-state transducer scanning right-to-left.

Formally Verified Results (sorry-free)

The following results are well-known in the literature. The proofs here sometimes differ from the classical ones, as certain constructions were adapted to be more amenable to formal verification in Lean 4.

1. Left-Independent $\leftrightarrow$ Regular Simulation

Given a left-independent CA C , construct a regular CA C' such that:

$$\Delta_{C'}^t(c)_i = \Delta_C^{2t}(c)_{i-t}$$

Conversely, given any CA C , construct a left-independent C' with $Q' = Q \cup (Q \times Q)$ such that:

$$\Delta_{C'}^{2t}(c)_i = \Delta_C^t(c)_{i+t}$$

This establishes the equivalence of OCA and CA up to a constant factor of 2 in time.

2. k -Step Left-Independent Speedup

Given a left-independent CA $C = (Q, \delta)$ and $k \geq 2$, construct a left-independent $C' = (Q^k, \delta')$ compressing k consecutive diagonal cells into one tuple. Define coordinate maps:

$$\psi(i, j) = ki + j, \quad \varphi(t, i, j) = t - (k - 1)i - j$$

Then for $i < 0$ and $0 \leq j < k$:

$$\text{comp}_{C'}(w, t, i)_j = \text{comp}_C(w, \varphi(t, i, j), \psi(i, j))$$

The proof proceeds by outer induction on t and inner descending induction on j within each time step. A variant without the quiescent-border assumption composes with the passive-border construction.

3. General k -Step RT Speedup

For any CA C and constant k , construct C' such that:

$$\text{trace}_{C'}(w)(i) = \text{trace}_C(w)(i + k)$$

This achieves a constant additive speedup by chaining PassiveBorder and DeadBorder constructions.

4. Passive Border for Left-Independent CAs

Given a left-independent CA C , construct C' whose border is **quiescent** ($\delta(\#, \#, \#) = \#$), while $\text{comp}_{C'} = \text{comp}_C$ inside the left-independent light cone. Together with Result 5, this shows that the unconstrained border in our formalization is without loss of generality.

5. Dead Border

Given any CA C , construct C' whose border state $\#$ is **dead** (absorbing: $\delta(\cdot, \#, \cdot) = \#$), while preserving the trace: $\text{trace}_{C'}(w)(t) = \text{trace}_C(w)(t)$ for all $t < c \cdot |w|$, where c is a constant depending on C' . In particular, the trace is preserved for any linear-time computation. Uses a zigzag folding of cells into lanes.

Advice Theory (Key Contribution)

The following results are likely **novel** and form the core contribution of this project. They develop a structural theory of *advice* for cellular automata, establishing closure properties of RT transducers and two-stage advice, and classifying prefix-stable RT-closed advice as RT transducers.

Result 1: RT transducers are closed under composition

has sorry

Given CA transducers $C_1 : \Sigma \rightarrow \Gamma_1$ and $C_2 : \Gamma_1 \rightarrow \Gamma_2$, there exists a CA C with $\text{trace_rt}_C = \text{trace_rt}_{C_2} \circ \text{trace_rt}_{C_1}$. This is the most technically challenging result in the project, requiring the full machinery of dead border, passive border, k -step speedup, and left-independent $\leftrightarrow$ regular simulation. The proof uses a multi-stage pipeline (AddBorder $\rightarrow$ CompressToDiag $\rightarrow$ SimFrom Λ $\rightarrow$ DecompressTriple $\rightarrow$ SpeedupKSteps). *Note: the main theorem is proven, but some intermediate pipeline stages still contain sorry (~8 total).*

Result 2: RT transducers are RT-closed

sorry-free modulo Result 1

If $f = \text{trace_rt}_C$ for some CA C , then f is RT-closed: $\mathcal{L}(\text{CA}_{\text{rt}}(\Sigma \times \Gamma)/f) = \mathcal{L}(\text{CA}_{\text{rt}}(\Sigma))$. This follows from Result 1: given a receiving CA C_r and an advice transducer C_a , one composes C_r with C_a (using Result 1) to obtain a single CA that simulates both.

Result 3: Prefix-membership advice is an RT transducer

sorry-free

For any $L \in \mathcal{L}(\text{CA}_{\text{rt}})$, the advice f_L defined by

$$f_L(w)_i = [w_{[0..i+1)} \in L]$$

is itself an RT transducer: $f_L = \text{trace_rt}_C$ for a suitable CA C that runs the recognizer for L and outputs the acceptance bit at each step. In particular, f_L is RT-closed (by Result 1).

Result 4: RT-closed $\wedge$ prefix-stable $\implies$ RT transducer*sorry-free*

If an advice f is both RT-closed and prefix-stable, then f is an RT transducer: $f = \text{trace_rt}_C$ for some CA C . The proof constructs C by observing that prefix-stability lets one reduce f to a product of prefix-membership advices (one for each output symbol $c \in \Gamma$, via the language $L_c = \{w \mid f(w)_{|w|} = c\}$), each of which is an RT transducer by Result 3. (The Lean code formally proves the two-stage version with the identity FST; the RT transducer statement is immediate.)

Definition: Two-Stage Advice

A **two-stage advice** is an advice $f : \Sigma^* \rightarrow \Gamma^*$ that factors as:

$$f = M \circ \text{trace_rt}_C$$

where C is a CA real-time transducer ($\Sigma \rightarrow B$) and M is a finite-state transducer scanning right-to-left ($B^* \rightarrow \Gamma^*$). Intuitively, the CA computes an intermediate annotation in real-time, and then a right-to-left FST post-processes it. This is a natural class: the CA captures the “global” left-to-right sweep inherent in real-time computation, while the FST captures bounded right-to-left look-ahead.

Result 5: Two-stage advice is closed under composition*sorry-free modulo Result 1*

Given two-stage advices $f_1 : \Sigma^* \rightarrow \Gamma_1^*$ and $f_2 : \Gamma_1^* \rightarrow \Gamma_2^*$, the composition $f_2 \circ f_1$ is again two-stage. The proof works by commuting the FST of f_1 past the CA of f_2 using a “backwards FSM” construction that absorbs the FST into the CA’s state space.

Result 6: Two-stage advice is RT-closed*sorry-free modulo Result 1*

If f is two-stage, then $\mathcal{L}(\text{CA}_{\text{rt}}(\Sigma \times \Gamma)/f) = \mathcal{L}(\text{CA}_{\text{rt}}(\Sigma))$. This follows from Result 1 (the CA component is RT-closed) and the fact that a right-to-left FST can be absorbed into the receiving CA.

Result 7: Middle advice is *not* two-stage*sorry-free*

The advice f_{mid} that marks position $\lfloor n/2 \rfloor$ (i.e., $f_{\text{mid}}(w)_i = [i = \lfloor |w|/2 \rfloor]$) cannot be expressed as a two-stage advice. The proof uses a bottleneck argument: the FST has finitely many states, but the middle position requires information about the full word length, which the CA’s real-time trace at the midpoint cannot encode in bounded state.

Open Question

Is every RT-closed advice two-stage, without the prefix-stability assumption? We conjecture that no such counterexample exists. However, if a non-two-stage RT-closed advice did exist, its RT-closedness proof would probably require a fundamentally different, non-geometric simulation construction — and it could be promising to investigate whether such a construction can be shown to be generally uncomputable.